



Instruction Learning Paradigms: A Dual Perspective on White-Box and Black-Box LLMs

Yanwei Ren^{1,2} · Liu Liu^{1,2} · Baosheng Yu³ · Jiayan Qiu⁴ · Quan Chen⁵

Received: 23 December 2025 / Revised: 24 April 2026 / Accepted: 28 July 2026

© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2026

Abstract

Optimizing instructions for large language models (LLMs) is critical for harnessing their full potential in complex and diverse tasks. However, relying solely on white-box approaches demands extensive computational resources and offers limited representational capacity, while black-box models can incur prohibitive financial costs. To address these challenges, we introduce a novel framework that seamlessly merges the strengths of both paradigms. Black-box models provide high-quality, diverse instruction initializations, and white-box models supply fine-grained interpretability through hidden states and output features. By enforcing a semantic similarity constraint, these components fuse into a unified high-dimensional representation that captures deep semantic and structural nuances, enabling an iterative optimization process to refine instruction quality and adaptability. The framework aligns black-box and white-box representations within a shared semantic space and incorporates a feature adaptation module to dynamically capture inter-model consistency and complementarity, thereby enhancing information fusion and semantic robustness. Extensive evaluations across a broad spectrum of benchmark tasks—ranging from complex reasoning to three English-to-X translation tasks—demonstrate that our approach consistently outperforms strong baselines on average. This fusion of black-box initialization with semantic refinement yields an effective and scalable solution for instruction optimization across diverse benchmark tasks.

Keywords Instruction learning · White-box LLMs · Black-box LLMs · Prompt optimization

Editor: Marco Lippi.

✉ Liu Liu
liuliubh@buaa.edu.cn

¹ School of Artificial Intelligence, Beihang University, Beijing, China

² Hangzhou International Innovation Institute, Beihang University, Hangzhou, China

³ Nanyang Technological University, Singapore, Singapore

⁴ University of Leicester, Leicester, UK

⁵ Kuaishou Technology, Beijing, China

1 Introduction

The application of large language models (LLMs) spans a diverse range of tasks and domains Naveed et al. (2023), highlighting the increasing importance of developing effective and high-quality instructions Zhang et al. (2024). While modern LLMs exhibit robust capabilities in instruction-following Heo et al. (2024); Wu et al. (2024); Li et al. (2023), the performance of these models is highly sensitive to the quality of the instructions they receive Dai et al. (2023); Cao et al. (2023). Poorly designed instructions can significantly limit their potential Lou et al. (2024). Even for powerful closed-source models, such as the ChatGPT series, the quality of input instructions directly impacts the output quality White et al. (2024). This is particularly evident in domains such as medical question-answering systems, where clear, concise, and precise instructions are crucial for minimizing the risk of misdiagnosis Hake et al. (2024). Thus, optimizing instructions is essential for unlocking the full potential of LLMs and achieving reliable, high-quality outcomes. Additionally, with supervised fine-tuning (SFT) demonstrating considerable success in enhancing LLM performance, the construction of high-quality instruction datasets has become a focal point of contemporary research Liu et al. (2024).

However, translating these research advances into robust real-world performance remains challenging Zhang et al. (2024); Xu et al. (2024). In particular, achieving effective instruction optimization across languages, tasks, and domains has emerged as a significant bottleneck. Instruction tuning can yield impressive zero-shot generalization on unseen tasks Wei et al. (2021), yet even state-of-the-art instruction-following models often struggle to maintain consistency across diverse or specialized tasks Ge et al. (2023). Cross-lingual adaptability is also limited by the predominantly English-centric nature of existing instruction data and models, which hinders performance in multilingual settings Chirkova and Nikoulina (2024). Instructions that perform well in one context may falter in another due to domain-specific nuances and task differences Ling et al. (2023), underscoring the limitations of purely labor-intensive prompt design. Addressing these cross-task, cross-domain, and cross-lingual generalization challenges is crucial for unlocking the full practical value of instruction optimization, as it would enable more consistent, high-quality outcomes across the diverse scenarios in which LLMs are deployed.

Traditional approaches to instruction optimization typically involve the manual refinement of prompts to guide models toward more accurate outputs Song et al. (2025). However, this method is both cost-prohibitive and difficult to scale, which has driven a shift towards automated approaches. Despite advancements in automated prompt engineering, several fundamental challenges persist. Current solutions often treat prompt optimization as a black-box problem Huang et al. (2024), relying on gradient-free search methods or heuristic-based techniques. These approaches are not only computationally expensive but also prone to converging at suboptimal local minima. Moreover, they frequently overlook the valuable intermediate representations within LLMs. Effectively leveraging these internal signals could offer more precise guidance throughout the optimization process. This raises a crucial question: how can we systematically integrate black-box feedback, such as evaluation scores, with white-box insights, such as hidden representations, to refine instructions and enhance performance? The core insight driving our approach is that large language models (LLMs) encode critical information regarding instruction quality within their hidden layers Liu et al. (2024). By integrating black-box feedback, such as scores from external

evaluation models, with white-box access to the internal hidden representations, we establish a more intelligent and informed feedback loop for instruction learning. This combined approach allows us to more effectively guide the optimization process. Furthermore, an iterative framework ensures that each optimization step is informed by up-to-date performance signals, thereby minimizing dependence on random or naive search strategies. The ultimate objective is to generate instructions that consistently produce higher-quality responses from black-box models while remaining adaptable across various application scenarios.

Therefore, we propose a novel instruction learning algorithm that consists of three key stages. As shown in Fig. 1. We firstly initialize the instruction dataset by harnessing the advanced capabilities of a cutting-edge black-box model, evaluating its performance and collecting hidden representations from a white-box model. Then, we train a custom neural network to predict instruction scores, encouraging the alignment of its representations with those of the highest-performing instructions. Finally, we iteratively refine the instructions by generating candidate prompts, extracting hidden features, and re-evaluating performance until convergence. This iterative process continues for a fixed number of iterations or until a predefined performance threshold is reached, ensuring steady convergence toward optimal instructions. The main contributions of our proposed framework are as follows:

- We propose a novel framework for instruction learning that effectively enhances instruction generation performance across various tasks, including black-box and white-box model, i.e., ChatGPT and LLaMA.
- We introduce a refined mechanism for task-specific adaptation that boosts performance on diverse linguistic and reasoning tasks, and incorporate a robust evaluation system, allowing for continuous refinement based on hidden representations and evaluation scores.
- Extensive experiments on a diverse set of tasks reveal that our framework consistently achieves the highest performance on several tasks, such as “periodic_elements” and “word_sorting”, as demonstrated in Table 2. The average score across 30 tasks demonstrates a clear improvement over existing methods, underscoring the effectiveness of our approach.

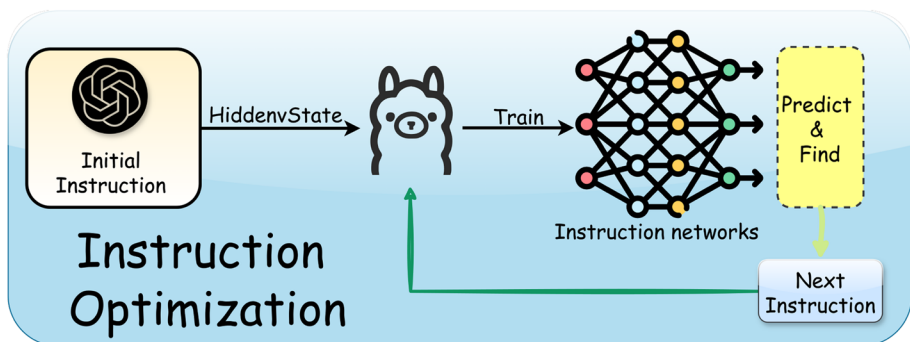


Fig. 1 The framework of instruction optimization based on black-box and white-box model

2 Related Works

2.1 Large Language Models and Instruction Tuning

With the continuous development and enhancement of computational resources, it has become possible to expand the number of parameters in pretrained models, enabling LLMs to achieve increasingly powerful capabilities Zhao et al. (2021); Hu et al. (2025). Taking the LLaMA model as an example, the number of parameters has increased from 65B (LLaMA 1) to 405B (LLaMA3.1) in just a few years Zhao et al. (2023). However, numerous studies have pointed out that simply increasing the number of model parameters no longer leads to significant performance improvements Hoffmann et al. (2022). With the advent of instruction tuning Zhao et al. (2021), it has been found that instruction tuning can address some mismatch issues Yu et al. (2023), allowing the model to better adapt to specific domains Kung and Peng (2023) or tasks Sun et al. (2023). Subsequent research has shifted the focus toward the construction of instruction datasetsZaleppa et al. (2024); Su et al. (2023); Wang et al. (2025). Many studies have used their own custom-built data sets to fine-tune large models that exhibit strong performance, such as LIMA Zhou et al. (2024), Vicuna Chiang et al. (2023), and WizardLLM Xu et al. (2023). These fine-tuned models have demonstrated exceptional capabilities.

2.2 Instruction Construction

The first method is manual constructionWen et al. (2024); Bai et al. (2024); Muhtar et al. (2024); Wu et al. (2024), which involves data that is either manually annotated or directly sourced from the internet. Creating these datasets typically does not involve machine learning techniques and relies entirely on manual collection and validationLi et al. (2025), making them relatively small in size. For example, data sets such as Super-Natural Instructions Wang et al. (2022) and Alpaca Taori et al. (2023) are high-quality data sets that have shown exceptional capabilities. However, the cost of manually constructing the data sets is very high and it is not feasible to scale them extensively. With the rise of ChatGPT, researchers have attempted to use ChatGPT Brown (2020) to replace humans in the dataset construction process. Based on ChatGPT, researchers have developed complex workflows to ensure the quality of the ChatGPT generated data sets Longpre et al. (2023). In addition to optimizing and constructing instruction data at the text level, some researchers have also explored the use of white-box large-language models to assist in instruction optimization and construction in high-dimensional spacesZheng et al. (2025); Bai et al. (2024). For example, InstructZero Chen et al. (2023) reformulates discrete instruction optimization as a low-dimensional soft-prompt optimization problem and applies Bayesian optimization to search for effective instructions for black-box LLMs. INSTINCT Lin et al. (2024) further extends this line of work by replacing the Gaussian-process surrogate with a neural-bandit-based neural surrogate and leveraging hidden representations from a white-box transformer, thereby improving the modeling of complex instruction-performance relationships. Different from these methods, which mainly rely on surrogate-guided latent-space search within a white-box framework, our method combines black-box initialization with white-box refinement, and further incorporates hidden/output representation fusion together with similarity-regularized iterative optimization.

2.3 Prompt Engineering

Prompt Engineering Hu et al. (2024) techniques offer a powerful alternative to manual prompt design in instruction learning by automatically generating and refining prompts through black-box search Jin et al. (2024). Systems such as Automatic Prompt Engineer treat prompt construction as a natural-language synthesis problem: an LLM proposes a pool of candidate instructions Cai et al. (2025), these are executed on a target model Sun et al. (2025), and the best-performing prompt is selected, often reaching human-level results and even uncovering superior chain-of-thought cues. Evolutionary variants like EvoPrompt maintain a population of candidates and iteratively mutate and select them Conti et al. (2018), efficiently exploring the discrete prompt space and achieving substantial gains over human-engineered and prior automatic methods. These strategies reduce tedious trial-and-error and adapt prompts to each task, frequently yielding higher-quality instructions without human intervention Feng et al. (2025). However, they can be computationally expensive due to the need to generate and evaluate many candidates, may overfit to specific datasets or models and thus generalize poorly Pan (2025), and, as black-box methods, can converge to suboptimal solutions. In practice, they complement other approaches: one can use an LLM to create diverse seeds, apply APE-style selection to pick winners, and integrate with white-box techniques in a hybrid pipeline that leverages the strengths of each to produce high-quality task-aligned instructions.

3 Methods

In this section, we introduce a novel instruction framework designed to improve the instruction learning through leveraging both black-box model and white-box model as shown in Fig. 2.

3.1 Black-Box Initialization

In previous methods Zhou et al. (2022); Chen et al. (2023); Lin et al. (2024) for instruction learning, most approaches relied on embedding soft prompts and examples into a white-box large language model to generate an initial set of instructions. However, this initialization process often led to several prominent issues. Specifically, the quality of the generated instructions heavily depended on the interpretative capabilities of the white-box model. Experimental results in Table 2 show that while white-box models could readily produce effective instructions for simpler tasks, their performance deteriorated significantly with more complex tasks, e.g., second word letter. This limitation can be attributed to the constrained parameter capacity and pretraining data of the white-box models, which often rendered them unable to fully comprehend the inputs during the initialization phase. As a result, ineffective initialization frequently occurred, leading to the failure of the entire instruction learning process.

With the continued development of black-box large language models (e.g., ChatGPT) and the decreasing costs of API usage, employing black-box models for initialization appears to be a more viable and effective solution. This approach not only addresses the limitations of white-box models but also mitigates the risk of catastrophic failure during

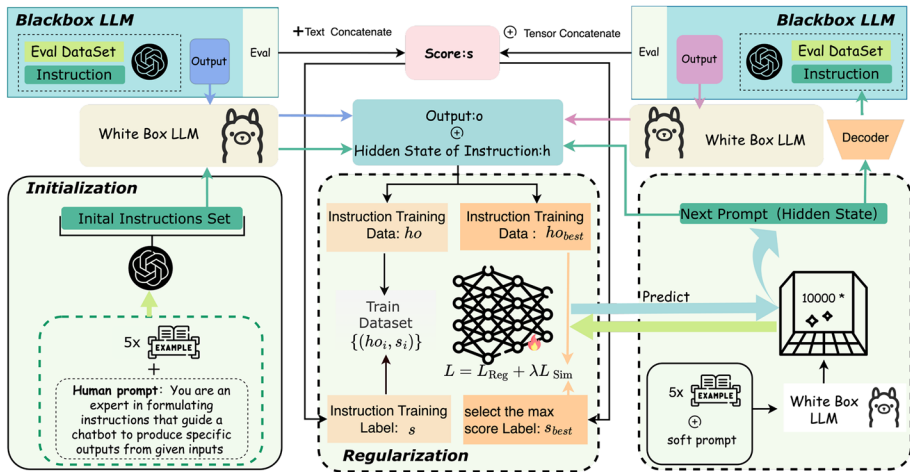


Fig. 2 Overview of our instruction learning framework. The framework integrates black-box initialization for generating diverse, high-quality instruction candidates with white-box refinement, which leverages hidden representations and additional output representations for a more comprehensive optimization process. A similarity constraint aligns the instructions with ChatGPT outputs, ensuring semantic consistency, while iterative updates further enhance instruction effectiveness across diverse tasks

the instruction learning process. In this study, we utilize a carefully crafted prompt, i.e., “You are an expert in formulating instructions that guide a chatbot to produce specific outputs from given inputs...”, along with a randomly selected set of five examples (comprising five input–output pairs) as input to ChatGPT. This approach enabled ChatGPT to generate an initial set of 40 instructions. By ensuring the quality of the initial instruction set while maintaining diversity among the instructions, we aim to provide the MLP with sufficient variability to learn the distinctions effectively. This variability allowed the model to better predict the scores associated with the corresponding soft prompts.

3.2 White-Box Instruction Representation

To capture a more comprehensive set of instruction characteristics and strengthen the optimization process, we propose a simple yet effective fusion strategy. This strategy combines the hidden representations extracted from intermediate layers of a white-box model with its corresponding output representations, yielding a unified representation. By integrating the advantages of both feature types, this unified representation provides a richer and more stable foundation for further optimization and analysis, which incorporate three representations.

Hidden Representation (h) The hidden representation is extracted from the intermediate layers of the white-box model, capturing the deeper semantic structure and underlying information of the input instructions. Formally, it can be obtained as:

$$h = \mathcal{M}_W(\pi), \quad (1)$$

where $\mathcal{M}_W$ denotes the function that extracts hidden features from the intermediate layers of the white-box model, and π denotes the input instruction.

Output Representation (o) The output representation directly reflects the generation result of the black-box model on the given input instruction, thereby encoding essential behavioral characteristics manifested during the execution process. It can be formulated as:

$$o = \mathcal{M}_W(\mathcal{M}_B(\pi)_{\text{output}}), \quad (2)$$

where $\mathcal{M}_B$ is the black-box model and $\mathcal{M}_B(\pi)_{\text{output}}$ means the final output of the black-box model.

Fusing the Representations To effectively harness the complementary information from both hidden and output representations, we first concatenate them into a single vector: $ho = [h; o]$, where $[h; o]$ denotes a dimensional concatenation of the two vectors. We then apply average pooling on the concatenated vector to reduce the feature dimensionality while smoothing out irrelevant fluctuations, resulting in a compact and robust fused representation:

$$\widehat{ho} = \text{AvgPool}(ho), \quad (3)$$

where $\text{AvgPool}(\cdot)$ is function of aggregating features from the hidden states of output and the instruction features by computing the mean over the specified dimensions.

3.3 Regularization Instruction Networks

Given the varying sizes of the model parameters, we assume that the instructions generated by ChatGPT are effective for most tasks. Since the instructions generated by ChatGPT are in textual form, and cosine similarity can effectively capture semantic similarity between texts, we introduce cosine similarity in the instruction learning process. The key is to maintain semantic consistency between instructions. Using cosine similarity, we can ensure that the generated instructions align in semantic direction with the effective instructions generated by ChatGPT, thus improving the quality and effectiveness of the instructions during the optimization process. This helps ensure that the neural network (NN) can explore the next instruction based on the features of the effective instructions generated by ChatGPT.

The neural network $\mathcal{N}$ predicts the reward score $\hat{s}$ and n is the number of instruction for a given standardized contextual embedding ho^* : $\hat{s} = \mathcal{N}(ho^*)$. The total loss $\mathcal{L}$ combines two components: the regression loss and the similarity loss:

$$\mathcal{L} = \mathcal{L}_{\text{Reg}} + \lambda \mathcal{L}_{\text{Sim}}, \quad (4)$$

where λ is the parameter, $\mathcal{L}_{\text{Reg}}$ and $\mathcal{L}_{\text{Sim}}$ are defined in Eqs. (5) and (6) as following

Regression Loss: The regression loss minimizes the error between the predicted scores $\hat{s}_i$ and the true rewards s_i^* , $i \in [N + T]$ and N is the number of initial instruction and T is total iterations:

$$\mathcal{L}_{\text{Reg}} = \frac{1}{N} \sum_{i=1}^N (\hat{s}_i - s_i^*)^2. \quad (5)$$

Similarity Regularization: To align contextual embeddings with the optimal instruction embedding, we calculate the cosine similarity between ho_i^* and ho_{best}^* : $\tilde{s}_i = \frac{\langle ho_i^*, ho_{\text{best}}^* \rangle}{\|ho_i^*\| \|ho_{\text{best}}^*\|}$

, where ho_{best}^* is the hidden state of the best performance instruction. The similarity loss encourages consistent alignment:

$$\mathcal{L}_{\text{Sim}} = \frac{1}{N} \sum_{i=1}^N (\sigma(\hat{s}_i) - \sigma(\tilde{s}_i))^2, \quad (6)$$

where $\sigma(\cdot)$ is the sigmoid function.

In order to dynamically update the parameters λ , we introduce the *Dynamic Weight Adjustment*. In particular, the weighting factor λ is dynamically adjusted to balance the loss components: $\lambda = \max(a, \min(b, \mathcal{L}_{\text{Reg}}))$, where a and b are two parameters that control the weight of similarity loss. The total loss $\mathcal{L}$ is minimized using the Adam optimizer, with gradients computed via backpropagation. After iterative updates, the trained model parameters are saved for evaluation.

Algorithm Description As shown in Algorithm 1, Our framework consists of initialization and training. Black-box models generate a diverse instruction set, while white-box models extract hidden representations to train a neural network for predicting instruction effectiveness. During training, the network iteratively refines soft prompts using black-box evaluation scores. A similarity constraint ensures semantic consistency, guiding optimization. The best instructions are continuously updated, balancing exploration and exploitation for improved instruction quality.

Input: Dataset $\mathcal{D}$, Initial instruction dataset $\{\pi_i\}_{i=1}^N$ (e.g. ChatGPT, GPT4, GPT4o, OpenAI-o1), Black-box model $\mathcal{M}_B$ (for score evaluation), White-box model $\mathcal{M}_W$ (for hidden representations), Maximum iterations T

Output: Best instruction π_{best}

- 1: **Initialization**
- 2: Obtain evaluate scores $\{s_i\}_{i=1}^N$ through the initial instruction dataset and dataset based on $\mathcal{M}_B$.
- 3: Extract hidden representations $\{h_i, o_i\}_{i=1}^N$ using $\mathcal{M}_W$, and combine h_i and o_i to form $[h_i; o_i]$ and construct the instruction training dataset $\{([h_i; o_i], s_i)\}_{i=1}^N$.
- 4: Train neural network $\mathcal{N}_s$ on the constructed instruction training dataset based on proposed loss function:
- 5: **Training**
- 6: **while** $t \leq T$ **do**
- 7: Predict the next latent soft prompt z_t using $\mathcal{N}_s$.
- 8: Generate instruction π_t using $\mathcal{M}_W$.
- 9: Retrain $\mathcal{N}_s$ with the updated dataset.
- 10: Obtain evaluate scores s_t through $\mathcal{M}_B$.
- 11: **if** $s_t > s_{\text{best}}$ **then**
- 12: Update $\pi_{\text{best}} \leftarrow \pi_t$ and $s_{\text{best}} \leftarrow s_t$.
- 13: **end if**
- 14: **end while**
- 15: **return** π_{best} .

Algorithm 1 Instruction learning algorithm under black-box model and white-box model

4 Experiments

4.1 Implementation

We utilized ChatGPT-4o as the initialization model during the instruction learning process. To obtain the hidden states corresponding to the instructions and their outputs, we employed Vicuna-13B as a white-box model and applied average pooling to integrate the hidden states of the two components. For directional constraints, dynamically updated weights were introduced to impose constraints during the neural network training process. Additionally, we leveraged ChatGPT-3.5-Turbo to generate outputs corresponding to the given instructions. All experiments were conducted on a single NVIDIA L20 GPU.

4.2 Datasets

The 30 datasets used for instruction-guided tasks in our study are the same as those in InstructZero Chen et al. (2023). We excluded two datasets, CS Algorithms and ASCII, as the test datasets for these tasks have not been made publicly available. The entire dataset consists of 30 tasks, which cover a wide range of practical applications, including semantic analysis, sentiment judgment, code modification, and word localization tasks. This comprehensive coverage ensures a broad representation of real-world applications.

4.2.1 Task Difficulty Calculation

To evaluate task difficulty in our benchmark, we compute difficulty scores based on multiple performance metrics across baselines. The final scores are shown in Table 1. Task difficulty is measured using three metrics: **Mean Error Difficulty**, which reflects the error rate across baselines where lower performance indicates higher difficulty; **Standard Deviation Difficulty**, which captures variability in performance, with higher variance suggesting greater difficulty; and **Combined Difficulty**, a weighted combination of the above metrics that balances error and variability:

$$D_{\text{combined}}^{(i)} = \alpha \cdot (1 - \mu^{(i)}) + \beta \cdot \sigma^{(i)}, \quad (7)$$

where

- $D_{\text{combined}}^{(i)}$: Combined difficulty score for the i -th sample/task.
- $\mu^{(i)}$: Mean score of the i -th sample across evaluations (higher mean implies lower error).
- $\sigma^{(i)}$: Standard deviation of scores for the i -th sample (quantifies performance variability).
- α, β : Weighting coefficients balancing error magnitude and variability (e.g., $\alpha = 0.7$, $\beta = 0.3$ in implementation).

Table 1 Computed task difficulty scores using different metrics

Task	Difficulty Score			Visual (Combined)
	Mean Error	Standard Deviation	Combined	
active_to_passive	0.0022	0.0031	0.0025	
antonyms	0.2021	0.0674	0.1593	
auto_categorization	0.7632	0.0291	0.5552	
auto_debugging	0.7114	0.0465	0.5238	
cause_and_effect	0.4188	0.0842	0.3678	
common_concept	0.8988	0.0209	0.6384	
diff	0.6306	0.1941	0.5425	
first_word_letter	0.0000	0.0000	0.0000	
informal_to_formal	0.6025	0.0442	0.4579	
larger_animal	0.8692	0.0476	0.6438	
letters_list	0.0842	0.2062	0.1038	
negation	0.2320	0.0155	0.1748	
num_to_verbal	0.0006	0.0011	0.0008	
object_counting	0.6178	0.0940	0.5084	
odd_one_out	0.4124	0.1297	0.3647	
orthography_starts_with	0.4660	0.0658	0.3919	
periodic_elements	0.8906	0.0332	0.6647	
rhymes	0.5418	0.3612	0.4624	
second_word_letter	0.4586	0.3405	0.4023	
sentence_similarity	0.9148	0.1422	0.6932	
sentiment	0.8924	0.0152	0.6399	
singular_to_plural	0.1960	0.2227	0.2053	
sum	0.3320	0.1654	0.3127	
synonyms	0.7555	0.0973	0.5782	
taxonomy_animal	0.4553	0.1652	0.4043	
translation_en-de	0.7362	0.0821	0.5825	
translation_en-es	0.6874	0.0598	0.5404	
translation_en-fr	0.7178	0.0690	0.5644	
word_sorting	0.5380	0.1172	0.4414	
word_unscrambling	0.4923	0.1073	0.4152	

(1) Bold indicates the maximum in each numeric column. (2) The gray bar encodes the Combined score scaled to [0, 1] for quick visual comparison

4.3 Results

We evaluate our method against several state-of-the-art approaches, including APE Zhou et al. (2022), InstructZero Chen et al. (2023), EvoPrompt Guo et al. (2023), and Instinct Lin et al. (2024), across a diverse set of 30 tasks. The results, summarized in Table 2, (Table 2 reports the accuracy of all compared methods on the 30-task benchmark, while the corresponding standard deviations across three random seeds are provided in Appendix A). demonstrate that our method consistently outperforms all baselines, achieving the highest mean score of **0.6955**, compared to **0.6639** for InstructZero and **0.6848** for EvoPrompt.

Table 2 Accuracy comparison over a benchmark of 30 tasks

Tasks & algorithms	APE Zhou et al. (2022)	InstructZero Chen et al. (2023)	EvoPrompt Guo et al. (2023)	Instinct Lin et al. (2024)	Ours
active_to_passive	1.0000	0.9967	0.9933	0.9967	1.0000
antonyms	0.6367	0.8267	0.7967	0.7300	0.7533
auto_categorization	0.2500	0.2567	0.2600	0.1900	0.2200
auto_debugging	0.2917	0.3750	0.3750	0.2500	0.2917
cause_and_effect	0.5733	0.8133	0.8267	0.7467	0.9467
common_concept	0.0691	0.0864	0.1211	0.0698	0.0956
diff	0.6733	0.6933	1.0000	0.5367	0.9300
first_word_letter	1.0000	1.0000	1.0000	1.0000	1.0000
informal_to_formal	0.5736	0.5310	0.6182	0.5098	0.5791
larger_animal	0.8967	0.9000	0.7933	0.9033	0.8567
letters_list	1.0000	0.5900	1.0000	0.9967	0.9933
negation	0.7533	0.7767	0.7867	0.7900	0.7633
num_to_verbal	0.9967	1.0000	1.0000	1.0000	1.0000
object_counting	0.3633	0.3600	0.1167	0.3300	0.3433
odd_one_out	0.6333	0.6133	0.6533	0.3600	0.6800
orthography_starts_with	0.4567	0.5067	0.6000	0.5800	0.5167
periodic_elements	0.9267	0.8667	0.9000	0.9400	0.9400
rhymes	0.1567	1.0000	0.6133	0.8567	0.6667
second_word_letter	0.7467	0.4333	0.4133	0.0667	0.9500
sentence_similarity	0.0000	0.0000	0.2833	0.0000	0.0000
sentiment	0.9133	0.8767	0.8767	0.9067	0.8767
singular_to_plural	1.0000	0.9867	1.0000	0.9967	0.9900
sum	0.6733	1.0000	1.0000	0.9900	0.9800
synonyms	0.3600	0.2767	0.1367	0.1267	0.1567
taxonomy_animal	0.3467	0.7167	0.7167	0.5300	0.6567
translation_en-de	0.8400	0.8233	0.8133	0.6600	0.7900
translation_en-es	0.8700	0.8733	0.8467	0.8733	0.8700
translation_en-fr	0.8867	0.8767	0.8833	0.7233	0.8433
word_sorting	0.3300	0.3100	0.5167	0.3267	0.5867
word_unscrambling	0.4400	0.5500	0.6033	0.3533	0.5900
Mean	0.6219	0.6639	0.6848	0.6113	0.6955

Each entry reports the mean accuracy achieved by a method on the corresponding task across three different random seeds (scaled to [0,1], with higher values indicating better performance). The corresponding standard deviations are reported separately in Table 7 in Appendix A. For APE, InstructZero, and EvoPrompt, we follow the previously reported three-seed benchmark statistics used in this comparison setting Lin et al. (2024). The MEAN row denotes the arithmetic average accuracy across all 30 tasks

We present a bubble chart visualization comparing task performance across APE, EvoPrompt, Instinct, InstructZero, and our proposed method. Each bubble represents a task, with the x-axis denoting task IDs, y-axis showing normalized scores (0–1), and size indicating task difficulty.

Distinct performance patterns emerge across baselines. APE exhibits high variance, excelling in some tasks while underperforming in others. EvoPrompt maintains stable performance, particularly in moderate-difficulty tasks. Instinct performs consistently but struggles with complex tasks requiring strong latent representations. InstructZero benefits

from adaptive refinement, though challenges persist in lower-scoring regions. Our method demonstrates improved performance across a wider range of tasks, achieving higher scores in more challenging cases, as evident in Fig. 3.

Across methods, higher task difficulty generally corresponds to greater performance fluctuations, highlighting the need for robust instruction optimization strategies balancing semantic adaptability and task-specific refinements. Our approach leverages both black-box initialization and white-box refinement, enabling more consistent optimization across diverse tasks.

Figure 4 shows the Normal Distribution Probability Graph for our method across three different seeds (0, 1, and 2). For comparison, we also include Instinct in the figure as the most directly relevant baseline for seed-level reference. The plots illustrate the cumulative percentage of scores relative to the performance scores, with reference lines indicating the expected distribution. For all three seeds, the majority of the scores for our method align closely with the reference line, suggesting that our method performs consistently and closely approximates the ideal distribution. The distribution of scores remains narrow, indicating stable performance with low variance across different initializations. This reinforces the robustness and reliability of our method. Our approach excels in reasoning-intensive tasks, such as *cause_and_effect*, where it achieves an impressive score of **0.9467**, significantly surpassing all other methods. Additionally, in simpler tasks like *first_word_letter* (1.0000) and *letters_list* (1.0000), our method attains perfect scores, highlighting its strong performance across varying levels of task difficulty.

To further assess the robustness of our approach, Fig. 5 (left) presents a ridgeline plot that illustrates the distribution of performance across tasks. Our method (denoted in green) consistently exhibits a higher density of high-performance scores, indicating not only superior performance but also greater consistency across a wide range of tasks. Figure 5 (right) depicts the evolution of instruction performance over iterations for selected tasks. While initial scores, generated by ChatGPT4o, are relatively low, the iterative optimization process significantly enhances performance, with our method surpassing the black-box LLM

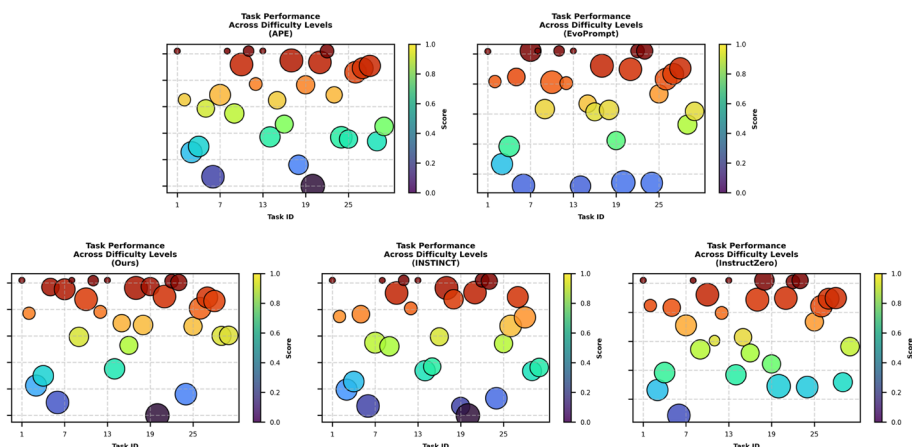


Fig. 3 Bubble chart comparison of task performance across different baselines: APE, EvoPrompt, INSTINCT, and InstructZero. The chart shows performance across tasks with varying difficulty levels, where bubble size represents task difficulty and color indicates performance scores

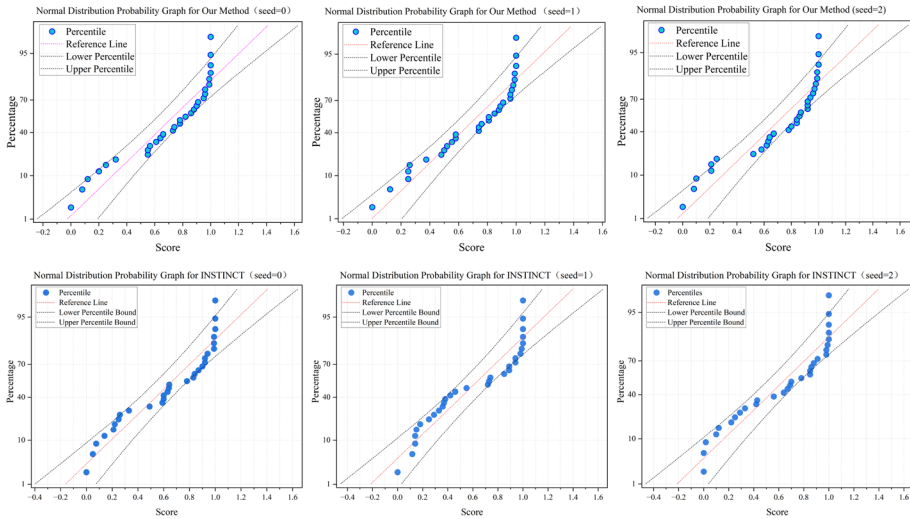


Fig. 4 Normal distribution probability graph for our method and instinct (3 seeds)

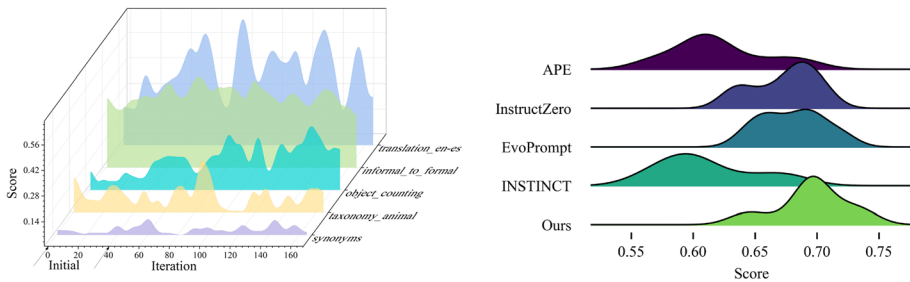


Fig. 5 (Left) Evolution of instruction performance across iterations for 5 tasks, and (Right) Ridgeline plot showing the score distributions of different methods across 30 tasks

in subsequent iterations. This underscores the efficacy of our iterative refinement process in improving performance over time.

Although our method achieves the best overall average performance, it does not improve uniformly across all tasks. In particular, the result on *sentence_similarity* suggests that the current framework is less effective for tasks requiring fine-grained semantic discrimination than for tasks with clearer structural or rule-based mappings. While our framework benefits from black-box models for strong instruction initialization, the subsequent prompt search and refinement process still depends on the representational quality of the white-box model, since candidate evaluation and optimization are carried out in the white-box-guided search space. Consequently, when the white-box model itself does not provide sufficiently reliable representations for subtle semantic distinctions, the downstream search process may also receive less reliable guidance, which can limit the effectiveness of refinement. By contrast, the proposed method tends to yield larger gains on tasks with clearer structural or symbolic mappings, such as *cause_and_effect*, *second_word_letter*, and *word_sorting*. These observations suggest that the current framework is more effective for tasks whose key distinctions

are more easily captured in the white-box representation space than for tasks requiring fine-grained semantic discrimination.

Overall, while our method exhibits some room for improvement in specific tasks such as stylistic transformation, it outperforms existing methods across a wide range of tasks. The superior performance in reasoning, syntactic transformation, and cross-lingual translation highlights the efficacy of our proposed method for multi-task instruction generation. These results demonstrate the potential of our approach to serve as a robust solution for diverse natural language processing tasks, providing a strong foundation for future improvements and extensions.

4.4 Efficiency Analysis

Table 4 compares the runtime, API cost, and GPU memory allocation of our method with Instinct. Our approach requires **643.57 min** of runtime, which is a **21.4% reduction** compared with Instinct (818.46 min). In terms of GPU memory usage, our method consumes **57.33 GB**, representing a **17.9% decrease** relative to Instinct (69.82 GB). These results demonstrate that our fusion-based framework is more resource-efficient and converges faster. Although the API cost of our method is slightly higher (\$3.24 vs. \$2.86), this modest increase is outweighed by the substantial savings in runtime and memory, making the proposed method a more practical and scalable solution for instruction optimization.

4.5 Ablation Experiment

In order to assess the impact of key components in our instruction learning framework, we conduct a series of ablation experiments. These experiments systematically evaluate the contributions of individual factors, including initialization strategies, similarity regularization, output representation. By selectively removing or modifying these components, we gain insights into their respective roles in shaping the overall performance of the model across a diverse set of tasks (Table 3 presents the main results, with additional details provided in Table 5. Specifically, Table 3 reports the accuracies obtained under different ablation settings on representative tasks).

The Validity of ChatGPT-Based Initialization We conduct an ablation study to evaluate the impact of black-box LLM initialization on instruction learning, comparing ChatGPT series (including version 3.5, 4, 4o) Openai and Chatgpt (2023) and DeepSeek V3 Liu et al. (2024). The results in Table 3 show that ChatGPT 4o achieves the highest mean score of 0.6713, surpassing ChatGPT 3.5 (0.5645) and ChatGPT 4 (0.6698). DeepSeek V3 performs well in certain tasks but has a lower overall mean (0.6299), suggesting that while different black-box models provide reasonable initializations, they lack consistency across tasks.

To further analyze the effectiveness of our approach, we integrate ChatGPT 4o initialization with white-box refinement and similarity constraints. Our method achieves a mean score of 0.6955, demonstrating a clear improvement over using ChatGPT alone. Notably, performance gains are most evident in structured reasoning tasks such as cause and effect (0.9467 vs. 0.8000 for ChatGPT 4o) and word unscrambling (0.5900 vs. 0.5533). These results confirm that while ChatGPT 4o provides strong initializations, our iterative refinement strategy enhances instruction quality beyond what black-box models alone can achieve. These findings highlight the complementary role of black-box initialization and

Table 3 Accuracy comparison across 10 representative tasks in ablation experiments

Tasks	Black-box LLM				Rep.	Loss	Ours
	ChatGPT	ChatGPT	ChatGPT	DeepSeek	Fuse Rep.	Sim. Const.	
	(3.5)	(4)	(4o)	(V3)			
active_to_passive	0.9833	0.9967	0.9633	1.0000	1.0000	0.9667	1.0000
antonyms	0.7667	0.7633	0.7533	0.7533	0.8000	0.7567	0.7533
cause_and_effect	0.6400	0.8800	0.8000	0.9200	0.9200	0.8133	0.9467
diff	0.6300	0.9967	0.9833	0.9600	0.9667	0.9567	0.9300
informal_to_formal	0.4331	0.4733	0.4714	0.4709	0.5720	0.4816	0.5791
object_counting	0.3700	0.2167	0.3700	0.4367	0.2600	0.3100	0.3433
orthography_starts	0.1200	0.5033	0.4567	0.1333	0.5267	0.6333	0.5167
sentiment	0.8767	0.8967	0.8600	0.8933	0.8867	0.9100	0.8767
translation_en-de	0.5433	0.8467	0.8267	0.0433	0.8033	0.8033	0.7900
word_unscrambling	0.4533	0.4133	0.5533	0.5467	0.5733	0.5133	0.5900
Mean (30 Tasks)	0.5645	0.6698	0.6713	0.6299	0.6717	0.6840	0.6955

The experiments evaluate the impact of different components and initialization settings, with ChatGPT 4o used as the initialization for our method. In the table header, “Rep.” denotes *representation*, “Fuse Rep.” denotes *fused output representation*, and “Sim. Const.” denotes *similarity constraint*

Table 4 Comparison of runtime, API cost, and GPU memory allocated

Method	Runtime [min]	API cost [\$]	GPU[GB]
INSTINCT	818.46	2.86	69.82
Ours	643.57	3.24	57.33

white-box optimization. By incorporating structural constraints and leveraging hidden representations, our approach systematically improves instruction effectiveness across diverse NLP tasks, reinforcing the necessity of a hybrid optimization framework over standalone black-box models.

Similarity regularization To evaluate the role of similarity regularization (SimReg) in instruction learning, we conduct an ablation study by removing the similarity constraint. The results indicate that the absence of SimReg leads to a noticeable degradation in instruction effectiveness. Tasks such as *cause_and_effect* (0.9467 $\rightarrow$ 0.8133) and *informal_to_formal* (0.5791 $\rightarrow$ 0.4816) exhibit significant drops, indicating that SimReg improves instruction alignment and enhances task adaptability. The mean performance decreases from 0.6955 to 0.6840, suggesting that SimReg contributes to better generalization by reinforcing consistency in learned representations. This aligns with the results in Fig. 6, where models with SimReg exhibit a more stable performance trajectory across iterations, and Fig. 7, which shows that similarity constraints help reduce variability in instruction scores. These findings demonstrate that SimReg is particularly beneficial in tasks requiring fine-grained distinctions, as it ensures the model retains high-quality instruction structures via learning.

Ablation Study on Output Representation We investigate the impact of additional output representation by comparing models trained with and without it. Panel b in Fig. 6 shows the performance difference over iterations, where models with additional output representation consistently outperform those without. This improvement is particularly evident in tasks involving complex patterns such as *letters_list*, *auto_categorization*, and *odd_one_out*. The green areas in the figures indicate where the model with output representation outperforms the model without it, confirming the stabilizing effect of the added representa-

Table 5 Performance comparison across different tasks with varying weight settings and neural network depth

Task	SimReg-weight			NN-depth		
	Weight = 0.01	Weight = 10^{-10}	Ours	Depth = 1	Depth = 3	Ours
active_to_passive	1.0000	1.0000	1.0000	0.9867	1.0000	1.0000
antonyms	0.7633	0.7467	0.7533	0.7700	0.7800	0.7533
auto_categorization	0.3333	0.3167	0.2200	0.1400	0.3200	0.2200
auto_debugging	0.3333	0.3333	0.2917	0.2917	0.3750	0.2917
cause_and_effect	0.9067	0.8667	0.9467	0.9467	0.8800	0.9467
common_concept	0.0868	0.0930	0.0956	0.0817	0.1133	0.0956
diff	0.9667	0.9700	0.9300	0.9667	0.6800	0.9300
first_word_letter	0.9967	0.9767	1.0000	0.9933	1.0000	1.0000
informal_to_formal	0.5144	0.5804	0.5791	0.4142	0.5245	0.5791
larger_animal	0.7500	0.8900	0.8567	0.7733	0.8500	0.8567
letters_list	1.0000	0.9967	0.9933	1.0000	0.9900	1.0000
negation	0.7833	0.8300	0.7633	0.8267	0.8600	0.7633
num_to_verbal	0.9933	0.9933	1.0000	0.9933	1.0000	1.0000
object_counting	0.2733	0.3567	0.3433	0.2867	0.3600	0.3433
odd_one_out	0.6133	0.6400	0.6800	0.6133	0.6600	0.6800
orthography_starts_with	0.6500	0.5733	0.5167	0.6200	0.6500	0.5167
periodic_elements	0.8800	0.9467	0.9400	0.9267	0.6400	0.9400
rhymes	0.4067	0.4567	0.6667	0.4167	0.1700	0.6667
second_word_letter	0.9167	0.7033	0.9500	0.7833	0.8900	0.9500
sentence_similarity	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
sentiment	0.8867	0.6467	0.8767	0.9000	0.9500	0.8767
singular_to_plural	0.9967	0.9767	0.9900	0.9700	1.0000	0.9900
sum	0.9733	0.9667	0.9800	0.8967	0.2900	0.9800
synonyms	0.1767	0.1867	0.1567	0.1133	0.1900	0.1567
taxonomy_animal	0.4567	0.7433	0.6567	0.4233	0.8300	0.6567
translation_en-de	0.8033	0.8167	0.7900	0.7967	0.8700	0.7900
translation_en-es	0.8500	0.8667	0.8700	0.8567	0.8700	0.8700
translation_en-fr	0.8433	0.8033	0.8433	0.8033	0.5800	0.8433
word_sorting	0.5967	0.6067	0.5867	0.6233	0.6300	0.5867
word_unscrambling	0.4733	0.4900	0.5900	0.5767	0.6651	0.5900
Mean	0.6742	0.6791	0.6955	0.6597	0.6651	0.6955

The last row reports the mean performance

tion. Furthermore, Table 3 further quantifies the impact of additional output representation. Removing output representation results in performance drops in tasks like *object_counting* (0.3433 to 0.2600) and *orthography_starts_with* (0.5267 to 0.5167), leading to a reduction in the overall mean score from 0.6955 to 0.6717. These results demonstrate that incorporating additional output representation helps the model leverage richer features, ensuring more stable and effective instruction optimization.

4.6 More Details on Ablation Experiment

Table 5 presents the performance comparison across different tasks with varying weight settings for SimReg and neural network depth. The results show the effect of different configu-

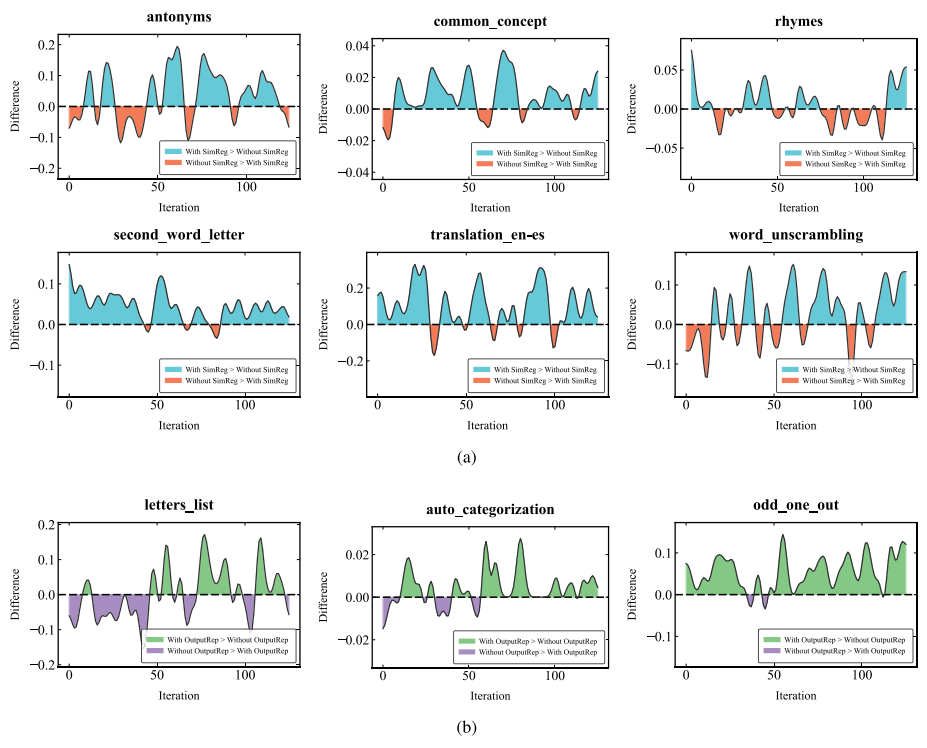
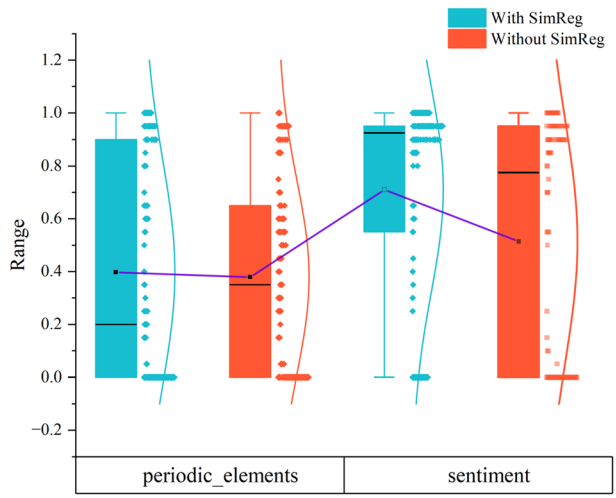


Fig. 6 Ablation studies: (a) effect of similarity regularization (SimReg); (b) effect of output representation

Fig. 7 Box plot comparing instruction optimization performance with and without similarity regularization (SimReg)



rations on task performance, highlighting the performance variation as the SimReg-weight and neural network depth are adjusted.

For SimReg-weight, with a value of 0.01, the method achieves strong performance across a variety of tasks, with notable scores in tasks like *active_to_passive* (1.0000), *first_*

word_letter (1.0000), and *letters_list* (1.0000). However, when the SimReg-weight is set to a very small value (10^{-10}), we observe a noticeable drop in performance for tasks such as *auto_categorization* (0.2200) and *cause_and_effect* (0.0956), indicating that a low SimReg-weight adversely affects performance in tasks that require fine-tuned semantic alignment.

In terms of neural network depth, we observe that increasing the depth from 1 to 5 generally leads to an improvement in performance across several tasks. For instance, *periodic_elements* increases from 0.9267 to 0.9400, and *word_unscrambling* improves from 0.5900 to 0.6233 when the network depth increases. This suggests that a deeper network helps capture more complex patterns and improves performance, particularly in tasks requiring higher-level reasoning and structural alignment.

The last row in Table 5 shows the mean performance across all tasks. Our method uses a depth of 5 and a dynamically adjusted SimReg weight, rather than a fixed number., achieves a mean score of 0.6955, which is the highest among the configurations, demonstrating the optimal combination of hyperparameters for task performance. In contrast, the lowest mean performance (0.6572) occurs when the SimReg-weight is set to 10^{-10} and depth is set to 1, indicating that both lower weight values and shallow network depths hinder the model's ability to generalize effectively across tasks.

The ablation results suggest that the effects of SimReg and network depth are not monotonic, but depend on how these components interact with task structure and search dynamics. For SimReg, fixed regularization strengths induce different trade-offs across tasks. A stronger similarity constraint can help stabilize optimization by keeping candidate instructions close to semantically promising regions, but it can also suppress useful exploration when larger deviations are needed to discover better instructions. This explains why different fixed weights are preferred by different tasks, while the dynamically adjusted SimReg used in the full model achieves the best overall mean performance. A similar pattern appears in the depth ablation. Increasing surrogate depth improves representational capacity, but it does not uniformly improve search quality. Since the surrogate is trained on relatively limited observations with sparse reward signals, deeper networks may also produce less stable rankings of candidate instructions across some tasks. This is particularly important because instruction optimization depends more on reliable ranking of candidates than on average regression quality alone. As a result, intermediate depth can underperform shallower settings on tasks such as *periodic_elements*, *rhymes*, *sum*, and *translation_en-fr*. These results suggest that stronger overall performance arises from a better balance between representation capacity, regularization, and optimization stability, rather than from any single component in isolation.

These results underline the importance of selecting appropriate hyperparameters for optimal task performance, while also indicating that the effects of SimReg and network depth depend on task characteristics rather than following a uniformly monotonic trend.

4.7 Generalization to a Different Target Model

To assess whether the proposed framework generalizes beyond the original ChatGPT-3.5-Turbo setting, we further replace the target model with GPT-4o-mini and evaluate all methods on the same 30-task benchmark. As shown in Table 6, our method still achieves the best overall mean score (0.7332), outperforming EvoPrompt (0.7118), InstructZero (0.6750), Instinct (0.6486), and APE (0.6220). Although the relative ranking varies across some indi-

Table 6 Performance comparison of different prompt optimization methods on a benchmark of 30 tasks, using GPT-4o-mini as the target model

Tasks & algorithms	APE	InstructZero	EvoPrompt	Instinct	Ours
active_to_passive	1.0000	1.0000	1.0000	0.9900	1.0000
antonyms	0.6260	0.8100	0.8200	0.8300	0.7986
auto_categorization	0.2400	0.3520	0.2700	0.2500	0.2863
auto_debugging	0.3000	0.3750	0.3750	0.2500	0.3200
cause_and_effect	0.5760	0.8260	0.8450	0.7670	0.9200
common_concept	0.0690	0.0920	0.2320	0.1250	0.2900
diff	0.6740	1.0000	0.9900	0.6720	0.8800
first_word_letter	1.0000	1.0000	1.0000	1.0000	1.0000
informal_to_formal	0.5320	0.5420	0.6360	0.5951	0.5100
larger_animal	0.8820	0.7900	0.7890	0.9200	0.9000
letters_list	0.9800	0.9900	1.0000	1.0000	1.0000
negation	0.7620	0.7850	0.8000	0.7800	0.8000
num_to_verbal	0.9900	1.0000	1.0000	0.9900	1.0000
odd_one_out	0.6330	0.6200	0.6620	0.4200	0.6800
object_counting	0.3820	0.2830	0.1800	0.2500	0.3600
orthography_starts_with	0.4630	0.5240	0.6600	0.6140	0.6600
periodic_elements	0.9510	0.9300	0.9200	0.9800	0.9600
rhymes	0.1620	0.7300	0.6160	0.8720	0.7670
second_word_letter	0.7650	0.3300	0.5200	0.1200	0.9800
sentence_similarity	0.0000	0.0000	0.2822	0.0000	0.0000
sentiment	0.9150	0.9100	0.9000	0.9000	0.9200
singular_to_plural	1.0000	0.9820	1.0000	1.0000	1.0000
sum	0.6630	1.0000	1.0000	0.9500	1.0000
synonyms	0.3600	0.3200	0.2600	0.2400	0.3100
taxonomy_animal	0.3520	0.8900	0.8500	0.5800	1.0000
translation_en-de	0.8400	0.8300	0.8600	0.8400	0.8200
translation_en-es	0.8620	0.8700	0.8400	0.8700	0.8900
translation_en-fr	0.9000	0.8300	0.8850	0.9100	0.8700
word_sorting	0.3500	0.2800	0.5400	0.3900	0.6800
word_unscrambling	0.4300	0.3600	0.6200	0.3537	0.3943
Mean	0.6220	0.6750	0.7118	0.6486	0.7332

The best result for each task is highlighted in bold

vidual tasks, the proposed method continues to show clear advantages on several tasks, indicating that the hybrid optimization framework is not tied to a single target model.

5 Conclusion

In this work, we propose a novel instruction learning framework that merges the strengths of black-box models (high-quality, diverse initializations) and white-box models (fine-grained interpretability). By enforcing a semantic similarity constraint, our method unifies hidden state representations to iteratively refine instruction quality. Extensive experiments across 30 tasks demonstrate consistent outperformance over state-of-the-art baselines, with an average performance of 0.6955. Looking ahead, we plan to reduce black-box dependencies

and extend the framework to broader instruction-generation scenarios, including reinforcement learning and automated prompt engineering. By bridging black-box robustness and white-box interpretability, this work offers a promising direction for next-generation LLM-driven applications.

Appendix A: Additional Statistical Results

To provide a clearer view of seed-level variability, Table 7 reports the standard deviations of accuracy across three different random seeds for all compared methods on the 30-task benchmark. The corresponding mean accuracies are reported in the main results table. Lower values indicate more stable performance across different runs.

Appendix B: Initialization Prompt

To guide ChatGPT in generating high-quality instructions, we utilized a structured initialization prompt. This prompt instructs the model to formulate 40 distinct instructions, each beginning with the phrase “The instruction was to,” while varying in style, structure, and complexity. The instructions must serve as high-level guiding principles without explicitly referencing input–output examples. Instead, they should be diverse in phrasing and sentence construction to ensure robustness and generalizability.

The Fig. 8 is the complete initialization prompt along with representative `{init_token}` examples from the *taxonomy_animal* task.

Appendix C: Best Instruction Across 30 Tasks

The Table 8 provides a comprehensive summary of the best-performing instructions identified for each of the 30 tasks in our evaluation. These instructions were discovered through an iterative optimization process, meaning they represent the prompt formulations that achieved the highest performance for their respective tasks. In each case, the instruction designated as “best” is the result of refining the prompt wording to maximally align the model’s output with the task’s objective. The tasks themselves span a broad range—from linguistic transformations and classification challenges to code execution and arithmetic operations—underscoring the versatility of our approach. The original results (summarized in Table VI) enumerate each task alongside its optimized instruction, illustrating how diverse tasks often require distinct prompt strategies to reach optimal performance.

The optimized instructions exhibit clear adaptation to the nature of each task through their wording and focus. For instance, tasks involving text transformation (such as converting an active-voice sentence to passive voice or informal language to formal) had instructions that explicitly stated the required transformation. The best instruction for the active-to-passive task directed the model to “rewrite the sentence to shift attention from the initiator of the action to the entity upon which the action is performed”, clearly describing

Table 7 Standard deviations of accuracy over a benchmark of 30 tasks across 3 random seeds

Tasks & algorithms	APE	InstructZero	EvoPrompt	Instinct	Ours
active_to_passive	0.0000	0.0027	0.0027	0.0058	0.0000
antonyms	0.1416	0.0072	0.0152	0.0557	0.0737
auto_categorization	0.0094	0.0119	0.0309	0.0436	0.0265
auto_debugging	0.0340	0.0000	0.0000	0.0000	0.0722
cause_and_effect	0.0891	0.0109	0.0475	0.1222	0.0231
common_concept	0.0207	0.0398	0.0000	0.0504	0.0244
diff	0.2667	0.2224	0.0000	0.3522	0.0361
first_word_letter	0.0000	0.0000	0.0000	0.0000	0.0000
informal_to_formal	0.0026	0.0024	0.0047	0.1163	0.0346
larger_animal	0.0054	0.0408	0.0791	0.0473	0.0821
letters_list	0.0000	0.1674	0.0000	0.0058	0.0058
negation	0.0109	0.0136	0.0027	0.0872	0.0351
num_to_verbal	0.0027	0.0000	0.0000	0.0000	0.0000
object_counting	0.0191	0.0929	0.0626	0.0000	0.1464
odd_one_out	0.0144	0.0871	0.0109	0.2458	0.0529
orthography_starts_with	0.1477	0.0871	0.0205	0.0436	0.0351
periodic_elements	0.0218	0.0606	0.0377	0.0400	0.0346
rhymes	0.0640	0.0000	0.0178	0.2223	0.1617
second_word_letter	0.2028	0.1872	0.2397	0.0764	0.0300
sentence_similarity	0.0000	0.0000	0.0357	0.0000	0.0000
sentiment	0.0144	0.0242	0.0054	0.0153	0.0351
singular_to_plural	0.0000	0.0109	0.0000	0.0058	0.0000
sum	0.2667	0.0000	0.0000	0.0100	0.0100
synonyms	0.0759	0.0925	0.0054	0.0231	0.0214
taxonomy_animal	0.2341	0.0838	0.1308	0.2138	0.2323
translation_en-de	0.0047	0.0098	0.0072	0.2615	0.0173
translation_en-es	0.0000	0.0054	0.0098	0.0153	0.0100
translation_en-fr	0.0027	0.0027	0.0027	0.1301	0.0208
word_sorting	0.0374	0.1143	0.0435	0.1888	0.0404
word_unscrambling	0.1389	0.0170	0.0072	0.1155	0.0173

Each entry reports the standard deviation corresponding to the mean accuracy shown in Table 2. For APE, InstructZero, and EvoPrompt, we follow the previously reported three-seed benchmark statistics used in this comparison setting Lin et al. (2024). Lower values indicate more stable performance across runs

the intended rephrasing. Similarly, the optimal prompt for a debugging task instructed the model to “use the Python interpreter to execute each line of code and print the output”, effectively guiding it through a step-by-step code execution procedure. These examples show how the highest-performing instructions encapsulate the exact operation or criterion needed for each task, whether it involves rephrasing text, performing a calculation, or simulating a process. By aligning the prompt’s content and style with the specific demands of the task, the instructions help focus the model’s reasoning on relevant actions and information, thereby improving task performance.

Across all tasks, a unifying characteristic of the optimized prompts is their clarity and specificity. Nearly all of the top instructions begin with a strong imperative verb (e.g., “Rewrite,” “Identify,” “Convert”) that immediately signals the required action to the model. This direct style ensures that the model’s output is closely aligned with the task goal, as the

Meta-instruction brief and {init_token} pairs

Role. You are an expert in crafting instructions that guide a chatbot to generate specific outputs from given inputs.

Task. Create 40 distinct instructions, each beginning with “The instruction was to”, and ensure substantial variation in style, structure, and length.

Constraints. Do not reference example inputs or outputs. Treat each line as a high-level guiding principle that, when paired with appropriate data, leads to the intended result. Avoid mentioning “inputs”, “outputs”, or showing explicit examples inside the instructions.

Reference pairs used as {init_token}:

Input: pink, shark, giraffe, truck, duck, monkey
Output: giraffe, monkey, shark, duck

Input: mango, camel, chicken, sock, frog, donkey
Output: donkey, chicken, camel, frog

Input: snail, chicken, potato, waiter, penguin, white
Output: chicken, snail, penguin

Input: snail, horse, green, blue, butterfly, scarf
Output: butterfly, horse, snail

Input: goat, pajamas, scarf, hippo, egg, whale
Output: hippo, whale, goat

Fig. 8 Initialization prompt and representative examples from the taxonomy_animal task

prompt precisely delineates what operation to perform. Furthermore, the instructions are concise yet detailed enough to avoid ambiguity. They often include explicit references to the key content or criteria of the task – for example, explicitly mentioning a “common category” to identify, the “first letter” to extract, or a “single-letter hint” to use in solving a puzzle. By highlighting such crucial details within the instruction, the prompts leave little room for misinterpretation, guiding the model along the intended solution path. This emphasis on unambiguous, task-focused language is consistent with best practices in prompt design and likely contributes substantially to their effectiveness.

Although all optimized instructions maintain clarity, they are not uniform in format; instead, their structural styles vary notably across different tasks. Some tasks are best addressed with a brief, direct command, whereas others benefit from a more elaborate, step-wise directive. For example, certain optimal prompts resemble a short procedure or pseudo-code, especially for tasks involving code execution or multi-step reasoning – a cue that a procedural tone can help the model follow the logical steps required. In contrast, simpler classification or conversion tasks often required nothing more than a single succinct sentence in the imperative mood to achieve top performance. This structural diversity suggests that the form of an effective instruction is highly task-dependent, with the prompt’s style being tailored to the nature of the problem. Importantly, however, even the more complex or structured instructions remain focused and unambiguous. In every case, the optimized prompt avoids extraneous information and zeroes in on the task, indicating that adapting the instruction’s form to the task can be done without sacrificing clarity. Such findings reinforce the idea that flexible prompt formulation – adjusting tone, detail, or format to fit the task – is key to eliciting the best possible performance from the model.

Several notable trends emerge when examining how these optimized instructions align with improved model performance. One clear pattern is the inclusion of domain-specific or

Table 8 Best-performing instructions for different tasks

Task	Best instruction
active_to_passive	Rewrite the sentence to shift attention from the initiator of the action to the entity upon which the action is performed
antonyms	Identify the logical inverse of the provided term and articulate it as a word
auto_categorization	Identify the common category or theme shared by the listed items
auto_debugging	Use the Python interpreter to execute each line of code and print the output
cause_and_effect	Highlight the statement that provides insight into the initial conditions affecting the plot
common_concept	Highlight a physical process that both items undergo
diff	Calculate the difference by executing a basic subtraction operation
first_word_letter	Output the first letter of each inputted word
informal_to_formal	Restructure the sentence to align with formal language standards
larger_animal	Determine which of the given options is more substantial in size and pick it
letters_list	Provide a spaced sequence of characters that reconstructs the original word faithfully
negation	Reject the factual claim in the sentence through the use of negation
num_to_verbal	Convert numbers in the range 1–100000 to their word form
object_counting	Count the number of items in the input and output the count
odd_one_out	Identify the singular element among a collection that fits a specific conceptual or thematic criterion
orthography_starts_with	Locate the term within the sentence that is associated with the single-letter hint
periodic_elements	Determine the element's name based on its location in the periodic chart
rhymes	Create a text that is as close to the original as possible, but with the given output
second_word_letter	Extract the second letter from the sequence and display it
sentence_similarity	Use the output of the previous model as a guide to determine the correctness of the generated text
sentiment	Give each input–output pair a positive or negative evaluation based on whether the output was an improvement or a worsening compared to the input
singular_to_plural	Create a program that takes a word as input and outputs its plural form
sum	Add the first number to the second number
synonyms	Create a program that takes input from the user and outputs a word that is similar in meaning to the input word
taxonomy_animal	Extract and sequentially arrange animal-related terms, omitting any non-animal words
translation_en-de	Translate the given input and output words into German, and create a new rule for each input–output pair
translation_en-es	Translate the words from English to Spanish
translation_en-fr	Identify the French equivalent for the provided English term
word_sorting	Place the words in order from the first letter of the alphabet to the last
word_unscrambling	Transform the scrambled input into a recognizable word

Each instruction represents the most effective formulation found during iterative optimization

task-specific cues directly in the instruction, which helps anchor the model's reasoning in the correct context. For example, the best prompt for a chemical knowledge task explicitly mentions the element's "location in the periodic chart," immediately pointing the model to consider periodic table information. Similarly, instructions for tasks dealing with letter positions or word puzzles explicitly reference those details: a prompt might instruct "output

the first letter of each word” or “provide a spaced sequence of characters” for reconstruction, ensuring the model knows to focus on letter-level operations. Another tactic observed is framing certain tasks as if they were programming or tool-use tasks. For instance, the optimized prompts for generating plural forms or synonyms asked the model to “create a program that takes a word as input and outputs” the transformed word. This approach effectively guides the model to produce a structured, single-word answer (as a program would), which is well-suited for tasks requiring a specific output format. These patterns indicate that performance gains were often achieved by making the instructions echo the structure of the solution or the process needed to obtain it. In other words, the prompt itself acts as a scaffold for the task: by leveraging the model’s strengths in following explicit procedural or rule-based instructions, the optimized prompts channel the model’s behavior towards the correct solution strategy. Such insights underscore how nuanced prompt formulation – down to including relevant context and mimicking procedural frameworks – can strongly influence the quality of an LLM’s output.

Acknowledgments This work was supported by National Natural Science Foundation of China: (No.62441617), National Key R&D Program of China under the Strategic Science and Technology Innovation Cooperation Special Project (No. 2025YFE0209100), and CCF-Kuaishou Fund for Exploring Large Models (No.CCFKuaiShou2025012).

Author Contributions Yanwei Ren wrote the main manuscript text, implemented the method, performed the experiments, and prepared all figures. Liu Liu supervised the project and revised the manuscript. All authors reviewed the manuscript.

Data Availability No datasets were generated or analysed during the current study.

References

- Bai, G., Chai, Z., Ling, C., Wang, S., Lu, J., Zhang, N., Shi, T., Yu, Z., Zhu, M., Zhang, Y., Song, X., Yang, C., Cheng, Y., & Zhao, L. (2024a). Beyond efficiency: A systematic survey of resource-efficient large language models. arXiv preprint arXiv:2401.00625 <https://doi.org/10.48550/arXiv.2401.00625>
- Bai, Y., Du, X., Liang, Y., Jin, Y., Zhou, J., Liu, Z., Fang, F., Chang, M., Zheng, T., Zhang, X., Ma, N., Wang, Z., Yuan, R., Wu, H., Lin, H., Huang, W., Zhang, J., Lin, C., Fu, J., Zhang, G. (2024b). COIG-CQIA: Quality is all you need for Chinese instruction fine-tuning. arXiv preprint arXiv:2403.18058 <https://doi.org/10.48550/arXiv.2403.18058>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://doi.org/10.48550/arXiv.2005.14165>
- Cai, W., Jiang, J., Wang, F., Tang, J., Kim, S., & Huang, J. (2025). A survey on mixture of experts in large language models. *IEEE Transactions on Knowledge and Data Engineering*, 37(7), 3896–3915. <https://doi.org/10.1109/TKDE.2025.3554028>
- Cao, Y., Kang, Y., Wang, C., & Sun, L. (2023). Instruction mining: Instruction data selection for tuning large language models. arXiv preprint arXiv:2307.06290 <https://doi.org/10.48550/arXiv.2307.06290>
- Chen, L., Chen, J., Goldstein, T., Huang, H., & Zhou, T. (2023). Instructzero: Efficient instruction optimization for black-box large language models. arXiv preprint arXiv:2306.03082.
- Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., & Xing, E. P. (2023, March 30). Vicuna: An open-source chatbot impressing GPT-4 with 90% ChatGPT quality. LMSYS Org <https://www.lmsys.org/blog/2023-03-30-vicuna/>
- Chirkova, N., & Nikoulina, V. (2024). Zero-shot cross-lingual transfer in instruction tuning of large language models arXiv preprint. arXiv:2402.14778.

- Conti, E., Madhavan, V., Petroski Such, F., Lehman, J., Stanley, K., & Clune, J. (2018). Improving exploration in evolution strategies for deep reinforcement learning via a population of novelty-seeking agents. *Advances in Neural Information Processing Systems*, 31 <https://proceedings.neurips.cc/paper/2018/hash/b1301141feffabac455e1f90a7de2054-Abstract.html>
- Dai, W., Li, J., Li, D., Tiong, A., Zhao, J., Wang, W., Li, B., Fung, P. N., & Hoi, S. (2023). Instructblip: Towards general-purpose vision-language models with instruction tuning. *Advances in Neural Information Processing Systems*, 36, 49250–49267.
- Feng, S., Fang, G., Ma, X., & Wang, X. (2025). Efficient reasoning models: A survey. *arXiv preprint arXiv:2504.10903*.
- Ge, Y., Hua, W., Mei, K., Ji, J., Tan, J., Xu, S., Li, Z., & Zhang, Y. (2023). Openagi: When llm meets domain experts. *Advances in Neural Information Processing Systems*, 36, 5539–5568
- Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., & Yang, Y. (2023). Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. *arXiv preprint arXiv:2309.08532*.
- Hake, J., Crowley, M., Coy, A., Shanks, D., Eoff, A., Kirmer-Voss, K., Dhanda, G., & Parente, D. J. (2024). Quality, accuracy, and bias in chatgpt-based summarization of medical abstracts. *The Annals of Family Medicine*, 22(2), 113–120.
- Heo, J., Xiong, M., Heinze-Deml, C., & Narain, J. (2024). Do llms estimate uncertainty well in instruction-following? *arXiv preprint arXiv:2410.14582*.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., ... Sifre, L. (2022). An empirical analysis of compute-optimal large language model training. *Advances in Neural Information Processing Systems*, 35, 30016–30030 <https://doi.org/10.52202/068431-2176>
- Huang, S., Yang, K., Qi, S., & Wang, R. (2024). When large language model meets optimization. *arXiv preprint arXiv:2405.10098*.
- Hu, L., Liu, Z., Zhao, Z., Hou, L., Nie, L., & Li, J. (2024). A survey of knowledge enhanced pre-trained language models. *IEEE Transactions on Knowledge and Data Engineering*, 36(4), 1413–1430. <https://doi.org/10.1109/TKDE.2023.3310002>
- Hu, Z., Zhang, Y., Xiao, M., Wang, W., Feng, F., & He, X. (2025). Exact and efficient unlearning for large language model-based recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 37(10), 5866–5877. <https://doi.org/10.1109/TKDE.2025.3594687>
- Jin, B., Liu, G., Han, C., Jiang, M., Ji, H., & Han, J. (2024). Large language models on graphs: A comprehensive survey. *IEEE Transactions on Knowledge and Data Engineering*, 36(12), 8622–8642. <https://doi.org/10.1109/TKDE.2024.3469578>
- Kung, P.-N., & Peng, N. (2023). Do models really learn to follow instructions? an empirical study of instruction tuning. *arXiv preprint arXiv:2305.11383*.
- Li, J., Liu, W., Ding, Z., Fan, W., Li, Y., & Li, Q. (2025). Large language models are in-context molecule learners. *IEEE Transactions on Knowledge and Data Engineering*, 37(7), 4131–4143. <https://doi.org/10.1109/TKDE.2025.3557697>
- Li, Z., Peng, B., He, P., & Yan, X. (2023). Evaluating the instruction-following robustness of large language models to prompt injection. *arXiv preprint arXiv:2308.10819*.
- Lin, X., Wu, Z., Dai, Z., Hu, W., Shu, Y., Ng, S.-K., Jaillet, P., & Low, B. K. H. (2024). Use your instinct: Instruction optimization for llms using neural bandits coupled with transformers. *Forty-first International Conference on Machine Learning*.
- Ling, C., Zhao, X., Lu, J., Deng, C., Zheng, C., Wang, J., Chowdhury, T., Li, Y., Cui, H., Zhang, X., Zhao, T., Panalkar, A., Mehta, D., Pasquali, S., Cheng, W., Wang, H., Liu, Y., Chen, Z., Chen, H., Zhao, L. (2025). Domain specialization as the key to make large language models disruptive: A comprehensive survey. *ACM Computing Surveys*, 58(3), 1–39 <https://doi.org/10.1145/3764579>
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., & Ruan, C. (2024b). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- Liu, B., Chen, C., Gong, Z., Liao, C., Wang, H., Lei, Z., Liang, M., Chen, D., Shen, M., Zhou, H., Jiang, W., Yu, H., & Li, J. (2024). MFTCoder: Boosting code LLMs with multitask fine-tuning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (pp. 5430–5441). Association for Computing Machinery <https://doi.org/10.1145/3637528.3671609>
- Longpre, S., Hou, L., Vu, T., Webson, A., Chung, H. W., Tay, Y., Zhou, D., Le, Q. V., Zoph, B., Wei, J., & Roberts, A. (2023). The Flan collection: Designing data and methods for effective instruction tuning. *Proceedings of the 40th International Conference on Machine Learning*, 202, 22631–22648 <https://proceedings.mlr.press/v202/longpre23a.html>
- Lou, R., Zhang, K., & Yin, W. (2024). Large language model instruction following: A survey of progresses and challenges. *Computational Linguistics*, 50(3), 1053–1095 https://doi.org/10.1162/coli_a_00523

- Muhtar, D., Li, Z., Gu, F., Zhang, X., & Xiao, P. (2024). Lhrs-bot: Empowering remote sensing with vgi-enhanced large multimodal language model. *European Conference on Computer Vision* (pp. 440–457). Springer.
- Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2023). A comprehensive overview of large language models. arXiv preprint. [arXiv:2307.06435](https://arxiv.org/abs/2307.06435)
- OpenAI: ChatGPT. May 24 version [Large language model] (2023). <https://chat.openai.com>
- Pan, H. (2025). Information Extraction From Scientific Literature. Temple University.
- Song, S., Li, X., Li, S., Zhao, S., Yu, J., Ma, J., Mao, X., Zhang, W., & Wang, M. (2025). How to bridge the gap between modalities: Survey on multimodal large language model. *IEEE Transactions on Knowledge and Data Engineering*, 37(9), 5311–5329. <https://doi.org/10.1109/TKDE.2025.3527978>
- Sun, X., Dong, L., Li, X., Wan, Z., Wang, S., Zhang, T., Li, J., Cheng, F., Lyu, L., Wu, F., & Wang, G. (2023). Pushing the limits of chatgpt on nlp tasks. arXiv preprint [arXiv:2306.09719](https://arxiv.org/abs/2306.09719).
- Sun, X., Shi, K., Tang, H., Wang, D., Xu, G., & Li, Q. (2025). Educating language models as promoters: Multi-aspect instruction alignment with self-augmentation. *IEEE Transactions on Knowledge and Data Engineering*, 37(8), 4564–4577. <https://doi.org/10.1109/TKDE.2025.3569585>
- Su, T., Zhang, J., Yu, Z., Wang, G., & Liu, X. (2023). Stkd: Distilling knowledge from synchronous teaching for efficient model compression. *IEEE Transactions on Neural Networks and Learning Systems*, 34(12), 10051–10064. <https://doi.org/10.1109/TNNLS.2022.3164264>
- Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., & Hashimoto, T. B. (2023, March 13). Alpaca: A strong, replicable instruction-following model. Stanford Center for Research on Foundation Models <https://crfm.stanford.edu/2023/03/13/alpaca>
- Wang, H., Li, W., Xia, X.-G., & Du, Q. (2025). Bihot: A large-scale dataset and benchmark for hyperspectral camouflaged object tracking. *IEEE Transactions on Neural Networks and Learning Systems*, 36(9), 16392–16406. <https://doi.org/10.1109/TNNLS.2025.3564059>
- Wang, Y., Mishra, S., Alipoormolabashi, P., Kordi, Y., Mirzaei, A., Arunkumar, A., Ashok, A., Dhanasekaran, A. S., Naik, A., Stap, D., Pathak, E., Karamanolakis, G., Lai, H. G., Purohit, I., Mondal, I., Anderson, J., Kuznia, K., Doshi, K., Patel, M., Pal, K. K., Moradshahi, M., Parmar, M., Purohit, M., Varshney, N., Kaza, P. R., Verma, P., Puri, R. S., Karia, R., Sampat, S. K., Doshi, S., Mishra, S., Reddy, S., Patro, S., Dixit, T., Shen, X., Baral, C., Choi, Y., Smith, N. A., Hajishirzi, H., & Khashabi, D. (2022). Super-naturalinstructions: Generalization via declarative instructions. on 1600+ nlp tasks. arXiv preprint <https://arxiv.org/abs/2204.07705>.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., & Le, Q. V. (2021). Fine-tuned language models are zero-shot learners arXiv preprint. [arXiv:2109.01652](https://arxiv.org/abs/2109.01652).
- Wen, B., Ke, P., Gu, X., Wu, L., Huang, H., Zhou, J., Li, W., Hu, B., Gao, W., Xu, J., Liu, Y., Tang, J., Wang, H., & Huang, M. (2024). Benchmarking complex instruction-following with multiple constraints composition. *Advances in Neural Information Processing Systems*, 37, 137610–137645
- White, J., Hays, S., Fu, Q., Spencer-Smith, J., Schmidt, D.C. (2024). ChatGPT Prompt Patterns for Improving Code Quality, Refactoring, Requirements Elicitation, and Software Design. In: Nguyen-Duc, A., Abrahamsson, P., Khomh, F. (eds) Generative AI for Effective Software Development. Springer, Cham. https://doi.org/10.1007/978-3-031-55642-5_4
- Wu, X., Wang, M., Liu, Y., Shi, X., Yan, H., Lu, X., Zhu, J., & Zhang, W. (2024b). Lifbench: Evaluating the instruction following performance and stability of large language models in long-context scenarios <https://doi.org/10.48550/arXiv.2411.07037> arXiv preprint.
- Wu, Z., Dadu, A., Nalls, M., Faghri, F., & Sun, J. (2024a). Instruction tuning large language models to understand electronic health records. *Advances in Neural Information Processing Systems*, 37, 54772–54786.
- Xu, C., Sun, Q., Zheng, K., Geng, X., Zhao, P., Feng, J., Tao, C., & Jiang, D. (2023). Wizardlm: Empowering large language models to follow complex instructions. arXiv preprint <https://arxiv.org/abs/2304.12244>.
- Xu, F. F., Song, Y., Li, B., Tang, Y., Jain, K., Bao, M., Wang, Z. Z., Zhou, X., Guo, Z., & Cao, M. (2024). Theagentcompany: benchmarking llm agents on consequential real world tasks. arXiv preprint <https://arxiv.org/abs/2412.14161>.
- Yu, Z., Zhang, X., Shang, N., Huang, Y., Xu, C., Zhao, Y., Hu, W., & Yin, Q. (2023). Wavecoder: Widespread and versatile enhanced instruction tuning with refined data generation. arXiv preprint. <https://arxiv.org/abs/2312.14187>
- Zaleppa, P., Kaza, S., & Taylor, B. (2024). Generating an instruction dataset to build cyber intelligent large language models. In: 2024 Cyber Awareness and Research Symposium (CARS), pp. 1–5. <https://doi.org/10.1109/CARS61786.2024.10778717>.
- Zhang, J., Vahidian, S., Kuo, M., Li, C., Zhang, R., Yu, T., Wang, G., & Chen, Y. (2024a). Towards building the federatedgpt: Federated instruction tuning. In: ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 6915–6919 IEEE.

- Zhang, Y.-F., Zhang, H., Tian, H., Fu, C., Zhang, S., Wu, J., Li, F., Wang, K., Wen, Q., Zhang, Z., Wang, L., Jin, R., & Tan, T. (2024b). Mme-realworld: Could your multimodal llm challenge high-resolution real-world scenarios that are difficult for humans? arXiv preprint <http://arxiv.org/abs/2408.13257>.
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., ... Wen, J.-R. (2023). A survey of large language models. arXiv preprint arXiv:2303.18223 <https://doi.org/10.48550/arXiv.2303.18223>
- Zhao, Z., Wallace, E., Feng, S., Klein, D., & Singh, S. (2021). Calibrate before use: Improving few-shot performance of language models. *International Conference on Machine Learning* (pp. 12697–12706) PMLR.
- Zheng, Y., Chen, Y., Qian, B., Shi, X., Shu, Y., & Chen, J. (2025). A review on edge large language models: Design, execution, and applications. *ACM Computing Surveys*, 57(8), 1–35.
- Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., & Levy, O. (2023). LIMA: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36, 55006–55021 <https://doi.org/10.52202/075280-2400>
- Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., & Ba, J. (2022). Large language models are human-level prompt engineers. arXiv preprint <http://arxiv.org/abs/2211.01910>.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.